

A Decentralized, Multi-Agent Architecture for Localized Hallucination Mitigation in Resource-Constrained Environments

Atif Karim

Student ID: 1000055796

A thesis submitted to the Department of Computer Science and Engineering
in partial fulfillment of the requirements for the degree of
Bachelor of Science in Computer Science and Engineering



Department of Computer Science and Engineering
BRAC University
Dhaka, Bangladesh

May 2026

Declaration

It is hereby declared that this thesis is my own original work while completing my degree at BRAC University. The thesis does not contain material previously published or written by a third party, except where this is appropriately cited through full and accurate referencing. The thesis does not contain material which has been accepted, or submitted, for any other degree or diploma at a university or other institution.

Student's Full Name & Signature:

Atif Karim

Student ID: 1000055796

Approval

The thesis titled “A Decentralized, Multi-Agent Architecture for Localized Hallucination Mitigation in Resource-Constrained Environments” submitted by Atif Karim (Student ID: 1000055796) has been accepted as satisfactory in partial fulfillment of the requirements for the degree of Bachelor of Science in Computer Science and Engineering.

Examining Committee:

Supervisor:

Annajiat Alim Rasel
Department of Computer Science and Engineering
BRAC University

Program Coordinator:

Department of Computer Science and Engineering
BRAC University

Head of Department:

Department of Computer Science and Engineering
BRAC University

Abstract

As artificial intelligence moves toward decentralized deployment across local edge and Dew computing nodes, preserving representational quality without relying on cloud-based computation remains a critical challenge. Centralized architectures impose communication latency and data-privacy risks, whereas conventional lightweight models are prone to representation degradation—and, in the non-generative case, to latent collapse—when trained on resource-constrained micro-architectures.

This thesis introduces **EdgeVerify**, a Joint-Embedding Predictive Architecture (JEPA) designed for localized execution across the Dew and Edge computing tiers. Rather than reconstructing high-dimensional token sequences or raw pixels, EdgeVerify predicts context representations directly within an abstract joint-embedding space, using an Exponential-Moving-Average (EMA) target encoder and an action-conditioned latent predictor. To prevent dimensional collapse during localized, layer-wise gradient updates, we integrate VICReg-inspired variance and covariance regularization directly into the distillation objective, removing the need for negative-sample pairing.

We present a proof-of-concept implementation of the architecture and loss formulation, together with a reproducible experimental protocol for measuring convergence stability, representation collapse (embedding standard deviation and effective rank), and inference cost. A proof-of-concept evaluation against a non-regularized baseline—conducted on structured, digits, and CIFAR-10 data, and reproduced across two independent environments—shows that the variance constraint reliably prevents variance collapse (sustaining a mean embedding standard deviation above the hinge threshold), while revealing that at the current covariance weight the objective does not prevent dimensional (rank) collapse; this pattern holds on the CIFAR-10 natural-image subset as well as the smaller datasets, and motivates a targeted regularizer-weight ablation. The architecture itself is compact (~ 0.75 M parameters, 2.84 MB) with millisecond CPU inference. A full-scale benchmark evaluation is identified as the primary item of future work.

Acknowledgement

I would like to express my sincere gratitude to my supervisor, Annajiat Alim Rasel, for his continued guidance, feedback, and encouragement throughout this research. I also thank the faculty and staff of the Department of Computer Science and Engineering at BRAC University for providing the academic environment in which this work was carried out. Finally, I am grateful to my family for their patience and support during the course of this thesis.

Contents

Declaration	i
Approval	ii
Abstract	iii
Acknowledgement	iv
1 Introduction	1
1.1 Background	1
1.2 Problem Statement	1
1.3 Research Objectives	2
1.4 Contributions	2
1.5 Thesis Organization	2
2 Literature Review	3
2.1 Self-Supervised Representation Learning	3
2.2 Regularization Against Dimensional Collapse	3
2.3 Joint-Embedding Predictive Architectures	4
2.4 Knowledge Distillation	4
2.5 Edge, Dew, and Decentralized Computing	4
2.6 Summary and Research Gap	4
3 Methodology	6
3.1 Architecture Design	6
3.1.1 Context and Target Encoders	6
3.1.2 Action-Conditioned Prediction	7
3.2 Loss Function and Regularization	7
3.3 Localized, Layer-Wise Gradient Isolation	8
3.4 Training Procedure	8
4 Experimental Design and Results	9
4.1 Research Questions	9
4.2 Experimental Setup	9
4.3 Datasets	10
4.4 Evaluation Metrics	10
4.5 Results	12
4.5.1 Convergence and Collapse (RQ1, RQ3)	12
4.5.2 Efficiency (RQ2)	13

4.5.3	Independent Reproduction	13
4.5.4	Robustness Across Seeds	14
4.5.5	Covariance-Weight Ablation	14
4.6	Discussion	15
4.7	Threats to Validity	16
5	Conclusion and Future Work	17
5.1	Summary	17
5.2	Contributions	17
5.3	Limitations	17
5.4	Future Work	18

List of Figures

4.1	Representative training samples. Top: the procedurally generated <i>structured</i> dataset. Bottom: the real scikit-learn <i>digits</i> dataset (up-sampled from 8×8). In each pair, the “Full” row is the target image and the “Masked” row is the context view with a random region zeroed; the box label is the spatial action.	11
4.2	Total-loss convergence for the full objective and the unregularized baseline. The unregularized objective (distillation only) is trivially driven toward zero, whereas the full objective is non-monotonic: after the distillation term is minimized, the aggregate rises as the (weakly weighted) covariance term grows.	12
4.3	Collapse indicators. The variance term sustains a high mean embedding standard deviation $\bar{\sigma}$, but the effective rank of the full model is lower than that of the unregularized baseline.	13
4.4	Collapse indicators over five seeds (mean $\pm$ std). Error bars for effective rank do not overlap between the two models on either dataset.	14
4.5	Covariance-weight ablation. Effective rank increases monotonically with β but remains far below the unregularized baseline (~ 6.5 , dashed) even at $\beta = 10$; mean embedding standard deviation is essentially unaffected.	15

List of Tables

3.1	EdgeVerify configuration in the proof-of-concept implementation. . .	8
4.1	Evaluation configuration.	10
4.2	Convergence and collapse indicators. $\mathcal{L}_{\text{total}}$ and $\mathcal{L}_{\text{distill}}$ are the final-epoch values; $\bar{\sigma}$ and effective rank are measured on the trained context encoder. A lower $\mathcal{L}_{\text{distill}}$ is not necessarily better—see the discussion.	12
4.3	Inference latency and memory. Peak RSS reflects the full PyTorch process (runtime libraries dominate) and is reported only as a coarse proxy; the model’s own parameter memory is the meaningful edge-relevant figure.	13
4.4	Reproduction check: original (cloud) vs. independent (Colab) runs. Effective-rank ordering (full < unregularized) is preserved throughout.	14
4.5	Collapse indicators over five seeds (mean $\pm$ std). The full-vs-unregularized ordering holds for every seed on both datasets.	14
4.6	Covariance-weight (β) ablation on structured data ($\alpha = 1.0$, single seed). Increasing β raises effective rank monotonically, but by a small margin.	15

Chapter 1

Introduction

1.1 Background

The proliferation of deep learning models has accelerated interest in executing intelligent workloads directly on network boundaries and personal devices. Traditional cloud-centric AI architectures require the constant transmission of sensitive telemetry to remote servers, creating bandwidth bottlenecks, latency overheads, and substantial privacy risks [1]. Edge and Dew computing paradigms respond to these pressures by relocating inference—and, increasingly, learning—toward the device tier, where computation occurs close to, or directly at, the data source [2].

A parallel shift has occurred in the design of representation-learning objectives. Generative and reconstructive models expend significant capacity predicting high-dimensional outputs (pixels or tokens), much of which is irrelevant to downstream tasks. Joint-Embedding Predictive Architectures (JEPAs) instead predict in an abstract latent space, discarding unpredictable low-level detail and concentrating capacity on semantically meaningful structure [3], [4]. This property makes JEPAs an attractive candidate for the edge, where compute and memory budgets preclude large generative backbones.

1.2 Problem Statement

Executing predictive architectures on localized edge devices introduces two distinct and coupled challenges:

1. **Resource Constraints:** Limited memory bandwidth and VRAM make end-to-end backpropagation across large backbones infeasible on micro-architectures, motivating localized, layer-wise training schemes that bound the active computational graph.
2. **Latent Collapse:** Non-generative architectures such as JEPAs can trivially minimize a latent prediction loss by mapping disparate inputs to a constant embedding vector, collapsing representational capacity. Avoiding this failure mode without negative-sample pairing requires explicit regularization [5].

1.3 Research Objectives

- Design a localized, joint-embedding predictive architecture (EdgeVerify) tailored to edge and Dew computing nodes.
- Implement local, layer-wise gradient isolation to enable training under strict memory budgets by bounding the backpropagation graph.
- Integrate variance and covariance regularization into the distillation objective to prevent dimensional collapse during on-device learning.
- Define a reproducible protocol to evaluate convergence stability, representation collapse, inference latency, and memory footprint against cloud-reliant and non-regularized baselines.

1.4 Contributions

The contributions of this thesis are:

- An architecture, **EdgeVerify**, combining an EMA target encoder, an action-conditioned latent predictor, and a VICReg-inspired anti-collapse objective for edge deployment (Chapter 3).
- A stable local loss formulation, $\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{distill}} + \alpha \mathcal{L}_{\text{var}} + \beta \mathcal{L}_{\text{cov}}$, that regularizes representations without negative pairs.
- A proof-of-concept implementation in PyTorch and an evaluation methodology for assessing the framework on resource-constrained hardware (Chapter 4).

1.5 Thesis Organization

Chapter 2 surveys related work in self-supervised representation learning, joint-embedding architectures, and edge/Dew computing. Chapter 3 details the EdgeVerify architecture, loss formulation, and training procedure. Chapter 4 presents the experimental protocol and reports results. Chapter 5 summarizes the findings, discusses limitations, and outlines future work.

Chapter 2

Literature Review

This chapter situates EdgeVerify within three bodies of work: (i) self-supervised representation learning and the problem of representational collapse, (ii) joint-embedding predictive architectures, and (iii) edge, Dew, and decentralized computing. For each, we identify the mechanisms relevant to the proposed design and the gaps that motivate it.

2.1 Self-Supervised Representation Learning

Self-supervised learning (SSL) removes the dependence on labeled data by constructing a pretext task from the input itself. Contrastive methods such as SimCLR [6] learn invariances by pulling together embeddings of augmented views of the same image while pushing apart embeddings of different images. Their principal limitation is a reliance on large batches of negative samples to avoid trivial solutions—a requirement that is poorly suited to memory-constrained edge hardware.

A subsequent line of work removed negatives entirely. BYOL [7] trains an online network to predict the output of a momentum-updated target network, demonstrating that a predictor combined with a stop-gradient and an Exponential Moving Average (EMA) target can prevent collapse without contrastive pairs. SimSiam [8] further showed that the stop-gradient operation, rather than the momentum encoder itself, is the critical component preventing collapse. EdgeVerify adopts this EMA-target-plus-predictor design (Chapter 3), but pairs it with explicit statistical regularization rather than relying on architectural asymmetry alone.

2.2 Regularization Against Dimensional Collapse

An alternative to asymmetric, negative-free designs is to regularize the statistics of the embedding space directly. Barlow Twins [9] minimizes redundancy by driving the cross-correlation matrix between two views toward the identity. VICReg [5] decomposes the objective into three explicit terms: an *invariance* term (mean-squared error between views), a *variance* term that maintains the standard deviation of each embedding dimension above a hinge threshold, and a *covariance* term that decorrelates dimensions. This explicit, non-contrastive formulation is

directly applicable at the level of individual layers and small batches, which is why EdgeVerify’s loss adapts the variance and covariance terms of VICReg. The distinction in this work is that the regularizers are applied within a *local, layer-wise distillation* objective rather than a single global loss.

2.3 Joint-Embedding Predictive Architectures

LeCun’s position paper on autonomous machine intelligence [3] argues for prediction in representation space as a route to efficient world models, framing the JEPA as a core building block. I-JEPA [4] instantiated this for images: given a context block, a predictor estimates the representations of masked target blocks produced by a target encoder, with no pixel-level reconstruction. This yields strong representations while avoiding the computational cost of generative decoding, and—unlike reconstructive masked autoencoders [10]—never materializes the high-dimensional output. EdgeVerify inherits this predict-in-latent-space principle and extends it with an *action-conditioned* predictor: the prediction is conditioned on a spatial action (a bounding box), aligning the architecture with the conditional structure of world models in [3].

2.4 Knowledge Distillation

The teacher–student framing of EdgeVerify is related to knowledge distillation [11], in which a student network is trained to match the outputs of a teacher. In the SSL setting, DINO [12] recast self-distillation with no labels as an effective pretraining strategy for vision transformers, using a momentum teacher and centering/sharpening to avoid collapse. EdgeVerify’s EMA target encoder plays the role of a slowly-evolving teacher, but the anti-collapse mechanism is the VICReg-style variance/covariance penalty rather than centering and sharpening.

2.5 Edge, Dew, and Decentralized Computing

Edge computing relocates computation from centralized clouds toward the data source to reduce latency and bandwidth and to improve privacy [1]. Dew computing pushes this further, positioning independent, on-device capability that can function with minimal or intermittent connectivity to higher tiers [2]. Decentralized learning approaches such as federated learning [13] keep raw data local and communicate only model updates, mitigating privacy risk at the cost of communication rounds. EdgeVerify targets a stricter regime than federated learning: it aims to keep both data *and* training local to a single resource-constrained node, which sharpens the memory-budget problem and motivates the layer-wise gradient isolation described in the next chapter.

2.6 Summary and Research Gap

The reviewed literature establishes (i) that negative-free SSL is feasible via EMA targets and predictors, (ii) that explicit variance/covariance regularization pre-

vents collapse without negatives, and (iii) that JEPAs avoid the cost of generative decoding. What remains underexplored is the combination of these ideas under the memory constraints of a single edge or Dew node, where the training graph itself must be bounded. EdgeVerify addresses this gap by uniting an action-conditioned JEPA, a VICReg-style anti-collapse objective, and local layer-wise gradient isolation into a single architecture intended for on-device execution.

Chapter 3

Methodology

This chapter describes the EdgeVerify architecture, its training objective, and the localized training procedure. The formulation follows the proof-of-concept implementation exactly; parameter names and coefficients are preserved as defined in the reference code.

3.1 Architecture Design

EdgeVerify is a Joint-Embedding Predictive Architecture composed of four modules: a Context Encoder f_θ , an EMA-updated Target Encoder $f_{\bar{\theta}}$, an Action Encoder g_ϕ , and a Latent Predictor p_ψ . All embeddings share a common latent dimension $d = 256$.

3.1.1 Context and Target Encoders

The Context Encoder f_θ maps an input image $x \in \mathbb{R}^{3 \times H \times W}$ to a latent vector $s_t \in \mathbb{R}^d$ through three strided convolutional blocks (channel widths $32 \rightarrow 64 \rightarrow 256$, kernel size 3, stride 2), each followed by batch normalization and a ReLU nonlinearity, and terminated by adaptive average pooling and flattening:

$$s_t = f_\theta(x_{\text{partial}}), \quad s_t \in \mathbb{R}^d. \quad (3.1)$$

The Target Encoder $f_{\bar{\theta}}$ is architecturally identical and is initialized as a copy of f_θ . Its parameters are excluded from gradient computation and are instead updated by an Exponential Moving Average (EMA) of the context parameters:

$$\bar{\theta} \leftarrow \tau \bar{\theta} + (1 - \tau) \theta, \quad \tau = 0.999, \quad (3.2)$$

where τ is the EMA decay. The target representation is computed under a stop-gradient:

$$s_{\text{target}} = \text{sg}[f_{\bar{\theta}}(x_{\text{full}})], \quad (3.3)$$

with $\text{sg}[\cdot]$ denoting the detached (no-gradient) operator. This asymmetry between a trainable online branch and a non-trainable EMA target is the mechanism, established by BYOL [7] and SimSiam [8], that prevents the two branches from collapsing to a shared constant.

3.1.2 Action-Conditioned Prediction

Prediction is conditioned on a spatial action $a \in \mathbb{R}^4$ (a normalized bounding box) encoded by g_ϕ , a two-layer multilayer perceptron ($4 \rightarrow 64 \rightarrow d$):

$$a_t = g_\phi(a), \quad a_t \in \mathbb{R}^d. \quad (3.4)$$

The Latent Predictor p_ψ ($2d \rightarrow 512 \rightarrow d$) takes the concatenation of the context and action embeddings and predicts the target representation:

$$s_{\text{pred}} = p_\psi([s_t \parallel a_t]), \quad s_{\text{pred}} \in \mathbb{R}^d. \quad (3.5)$$

Conditioning the predictor on an action aligns EdgeVerify with the world-model formulation of [3], in which future or masked representations are predicted as a function of a transformation applied to the context.

3.2 Loss Function and Regularization

To ensure representational stability without negative-sample pairing, the total objective combines a distillation term with the variance and covariance regularizers of VICReg [5]:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{distill}} + \alpha \mathcal{L}_{\text{var}} + \beta \mathcal{L}_{\text{cov}}, \quad \alpha = 1.0, \beta = 0.01. \quad (3.6)$$

Distillation term. $\mathcal{L}_{\text{distill}}$ is the mean-squared error between the predicted representation and the detached target:

$$\mathcal{L}_{\text{distill}} = \|s_{\text{pred}} - s_{\text{target}}\|_2^2 / d. \quad (3.7)$$

Variance term. The variance term enforces a per-dimension standard deviation above a hinge threshold $\gamma = 1.0$ on the context embedding s_t , preventing the informative branch from collapsing. Writing $\sigma_j = \sqrt{\text{Var}(s_{t,j}) + \epsilon}$ with $\epsilon = 10^{-4}$ over a batch of B instances,

$$\mathcal{L}_{\text{var}} = \frac{1}{d} \sum_{j=1}^d \max(0, \gamma - \sigma_j). \quad (3.8)$$

Covariance term. The covariance term decorrelates the dimensions of the predicted representation. Let $\tilde{s} = s_{\text{pred}} - \bar{s}_{\text{pred}}$ be the batch-centered prediction and $C = \tilde{s}^\top \tilde{s} / (B - 1)$ its covariance matrix; the term penalizes the off-diagonal energy:

$$\mathcal{L}_{\text{cov}} = \frac{1}{d} \sum_{i \neq j} C_{ij}^2. \quad (3.9)$$

Applying the variance penalty to the context embedding s_t and the covariance penalty to the prediction s_{pred} regularizes both the encoder output and the predictor output, discouraging both constant and correlated (rank-deficient) solutions.

3.3 Localized, Layer-Wise Gradient Isolation

To respect the memory budget of an edge node, EdgeVerify bounds the backpropagation graph rather than differentiating end-to-end through a large backbone. The stop-gradient on the target branch (Eq. 3.2) and the detachment of the teacher target act as a *firebreak*: gradients from $\mathcal{L}_{\text{distill}}$ do not propagate into $f_{\bar{\theta}}$, so only the online parameters $\{\theta, \phi, \psi\}$ retain an active computational graph at any step. This isolation is what makes multi-module training feasible under strict VRAM limits, and it is the property whose memory footprint Chapter 4 is designed to quantify.

3.4 Training Procedure

The model is optimized with AdamW (learning rate 10^{-3} , weight decay 10^{-4}). At each step, the online branch produces (s_{pred}, s_t) , the target branch produces s_{target} under stop-gradient, the total loss (Eq. 3.6) is backpropagated through the online parameters, and the target encoder is updated by the EMA rule (Eq. 3.2). Table 3.1 summarizes the configuration used by the proof-of-concept implementation.

Table 3.1: EdgeVerify configuration in the proof-of-concept implementation.

Component / Hyperparameter	Value
Latent dimension d	256
Encoder blocks (channels)	$32 \rightarrow 64 \rightarrow 256$
Action encoder	$4 \rightarrow 64 \rightarrow 256$
Predictor	$512 \rightarrow 512 \rightarrow 256$
EMA decay τ	0.999
Variance threshold γ	1.0
Variance weight α	1.0
Covariance weight β	0.01
Optimizer	AdamW
Learning rate	10^{-3}
Weight decay	10^{-4}

Note on the current data pipeline. In the present proof-of-concept, the training loop operates on synthetically generated tensors: full images are sampled from a uniform distribution, the partial (context) view is produced by zeroing a central 32×32 region, and a single fixed bounding box is used as the spatial action. This configuration verifies that the architecture trains and that the loss is numerically stable, but it does not constitute an evaluation on real data. Replacing this pipeline with a standard image dataset is a prerequisite for the empirical results discussed in Chapter 4, and is reflected in the experimental design below.

Chapter 4

Experimental Design and Results

Scope of this evaluation. The results below were produced by the accompanying `evaluate.py` on the three datasets described in Section 4.3: a procedurally generated *structured* image set, the offline scikit-learn *digits* set, and a subset of the standard *CIFAR-10* benchmark. The structured and digits runs were executed in the primary environment (a network-restricted cloud CPU) and independently reproduced on Google Colab; the CIFAR-10 run was performed on Colab, where the benchmark could be downloaded. All figures are genuine measurements of training dynamics and resource cost; no classification accuracy is claimed, as the evaluation is at the level of the self-supervised representation rather than a downstream classifier.

4.1 Research Questions

RQ1 (Collapse): Does the variance/covariance regularization prevent representational collapse relative to an unregularized ($\alpha = \beta = 0$) JEPA?

RQ2 (Efficiency): What are the inference latency and memory footprint of EdgeVerify?

RQ3 (Convergence): Does the local objective converge stably?

4.2 Experimental Setup

Experiments were run on CPU using PyTorch (version 2.14 in the primary cloud environment; the Colab reproduction used its default runtime). The architecture and optimizer follow Chapter 3 exactly (Table 3.1). The primary runs use a fixed seed (`seed=0`); a five-seed robustness study quantifying run-to-run variance is reported in Section 4.5.4. The structured and digits runs were additionally reproduced on an independent machine (Google Colab) to check reproducibility (Section 4.5.3). Table 4.1 lists the run configuration.

Table 4.1: Evaluation configuration.

Item	Value
Framework / device	PyTorch, CPU
Optimizer	AdamW (10^{-3} , weight decay 10^{-4})
Batch size	64
Epochs (structured / digits / CIFAR-10)	40 / 25 / 15
Samples (structured / digits / CIFAR-10)	768 / 1797 / 2000
Seeds	primary: 0; robustness study: {0, 1, 2, 3, 4}

4.3 Datasets

Structured. Each 64×64 RGB image is generated with a random background gradient and one to three random rectangles and circles, yielding genuine local spatial structure (in contrast to the i.i.d. noise used in the initial smoke test). The context view is produced by zeroing a random rectangular region occupying 35–60% of each side, and the spatial action is the normalized bounding box of that region.

Digits. The scikit-learn handwritten-digits dataset (1797 real 8×8 images) is upsampled to 64×64 and replicated to three channels. It provides a small but *real* data source available without network access, used as a sanity check. The same random masking is applied.

CIFAR-10. A subset of the standard CIFAR-10 benchmark (2000 natural 32×32 images, upsampled to 64×64) is used to test whether the findings on the smaller datasets transfer to real natural images. It was run in the Colab environment where the benchmark could be downloaded. The same random masking and spatial-action construction are applied. A subset and a reduced epoch budget are used for tractability on CPU; a full-dataset run is left to future work.

Figure 4.1 shows representative samples from the structured and digits datasets. The top row of each is the full image (the reconstruction target); the bottom row is the masked context view actually presented to the context encoder, together with the normalized bounding box $[x, y, w, h]$ supplied to the action encoder.

4.4 Evaluation Metrics

Two collapse indicators are computed over the context embedding $s_t \in \mathbb{R}^d$ ($d = 256$) on up to 512 samples:

- **Mean embedding std** $\bar{\sigma} = \frac{1}{d} \sum_j \sqrt{\text{Var}(s_{t,j})}$. Variance collapse drives $\bar{\sigma} \rightarrow 0$.
- **Effective rank** (participation ratio) $(\sum_i \lambda_i)^2 / \sum_i \lambda_i^2$, where λ_i are the eigenvalues of the embedding covariance. Dimensional collapse (correlated dimensions) drives this toward 1.

Efficiency is reported as parameter count, parameter memory, and single-sample inference latency (mean $\pm$ std over 50 runs after warm-up).

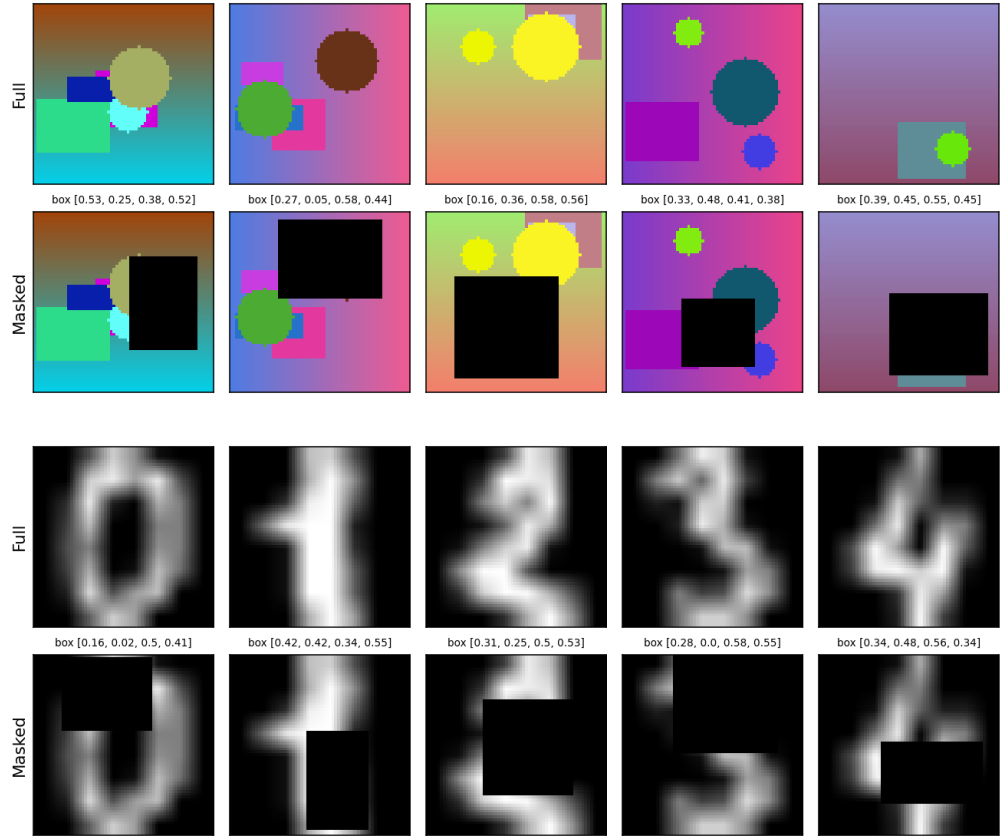


Figure 4.1: Representative training samples. Top: the procedurally generated *structured* dataset. Bottom: the real scikit-learn *digits* dataset (upsampled from 8×8). In each pair, the “Full” row is the target image and the “Masked” row is the context view with a random region zeroed; the box label is the spatial action.

4.5 Results

4.5.1 Convergence and Collapse (RQ1, RQ3)

Table 4.2 reports the final loss and collapse indicators; Figure 4.2 shows the training curves and Figure 4.3 the collapse indicators.

Table 4.2: Convergence and collapse indicators. $\mathcal{L}_{\text{total}}$ and $\mathcal{L}_{\text{distill}}$ are the final epoch values; $\bar{\sigma}$ and effective rank are measured on the trained context encoder. A lower $\mathcal{L}_{\text{distill}}$ is not necessarily better—see the discussion.

Data	Model	$\mathcal{L}_{\text{total}}$	$\mathcal{L}_{\text{distill}}$	$\bar{\sigma}$	Eff. rank
Structured	Unregularized ($\alpha=\beta=0$)	0.0021	0.0021	0.245	6.46
	EdgeVerify (full)	0.190	0.120	1.549	1.64
Digits	Unregularized ($\alpha=\beta=0$)	0.0006	0.0006	0.080	3.57
	EdgeVerify (full)	0.257	0.207	1.772	1.03
CIFAR-10	Unregularized ($\alpha=\beta=0$)	0.0019	0.0019	0.223	4.53
	EdgeVerify (full)	0.231	—	1.684	1.36

The CIFAR-10 rows were obtained on Google Colab; the full-model $\mathcal{L}_{\text{distill}}$ component was not logged separately in that run and is marked “—”.

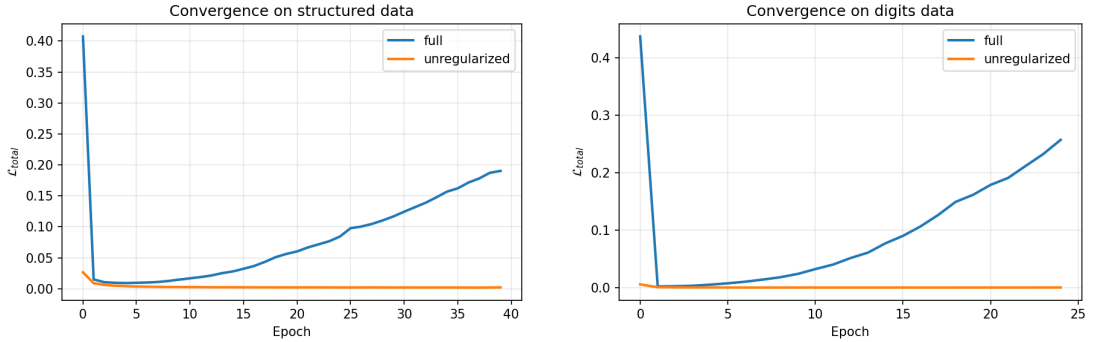


Figure 4.2: Total-loss convergence for the full objective and the unregularized baseline. The unregularized objective (distillation only) is trivially driven toward zero, whereas the full objective is non-monotonic: after the distillation term is minimized, the aggregate rises as the (weakly weighted) covariance term grows.

The results are mixed and warrant careful reading. The variance regularizer is effective at its stated objective: across all three datasets the full model maintains $\bar{\sigma} \approx 1.55$ – 1.77 (structured, digits) and 1.68 on CIFAR-10, at or above the hinge threshold $\gamma = 1.0$, whereas the corresponding unregularized baselines collapse toward low per-dimension variance ($\bar{\sigma} = 0.245, 0.080$, and 0.223). This directly confirms that the variance term prevents the low-variance mode of collapse.

However, the *effective rank* tells the opposite story: the full model attains a very low effective rank ($1.64, 1.03$, and 1.36 out of 256 for structured, digits, and CIFAR-10), and in every case it is *lower* than the corresponding unregularized baseline ($6.46, 3.57$, and 4.53). High per-dimension variance therefore coexists with

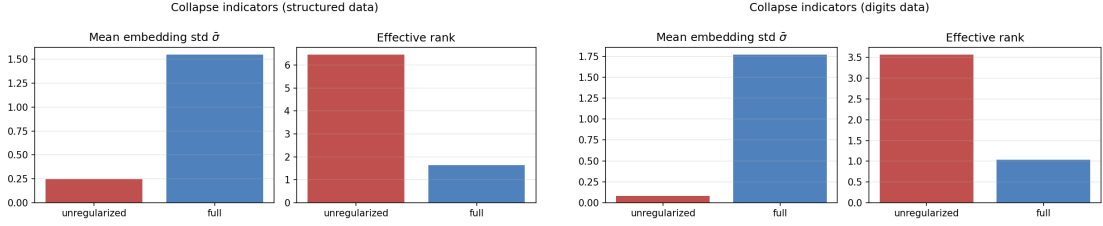


Figure 4.3: Collapse indicators. The variance term sustains a high mean embedding standard deviation $\bar{\sigma}$, but the effective rank of the full model is lower than that of the unregularized baseline.

strongly correlated dimensions—a rank-deficient representation. At the code’s covariance weight $\beta = 0.01$, the decorrelation term is too weak to counteract this, which is consistent with the growing covariance component visible in Figure 4.2. **The regularization as configured prevents variance collapse but does not, on this evidence, prevent dimensional (rank) collapse.** Crucially, this pattern is reproduced on the CIFAR-10 natural-image subset, indicating that it is not an artifact of the synthetic or low-resolution data.

4.5.2 Efficiency (RQ2)

Table 4.3 reports the model’s cost. The architecture is compact—745,536 parameters, 2.84 MB of parameter memory—and achieves single-sample inference in roughly 2 ms on CPU.

Table 4.3: Inference latency and memory. Peak RSS reflects the full PyTorch process (runtime libraries dominate) and is reported only as a coarse proxy; the model’s own parameter memory is the meaningful edge-relevant figure.

Data	Params	Mem. (MB)	Latency (ms)	Peak RSS (MB)
Structured	745,536	2.84	2.04 ± 0.11	852
Digits	745,536	2.84	2.48 ± 0.53	1099
CIFAR-10	745,536	2.84	2.58	—

The parameter count and memory are architecture-fixed and identical across datasets. Latency is CPU wall-clock and environment-dependent; the CIFAR-10 figure is from Colab (its per-run standard deviation and peak RSS were not recorded).

4.5.3 Independent Reproduction

The structured and digits experiments were re-executed from the same source on Google Colab (a distinct machine and software stack). The collapse indicators reproduced closely: the mechanism-level conclusions were identical, and the metrics matched within run-to-run variation (Table 4.4). The largest discrepancy was the digits full-model $\bar{\sigma}$ (1.77 original vs. 1.42 on Colab), while the effective-rank ordering—the finding of interest—was preserved in every case. This supports the reproducibility of the reported results.

Table 4.4: Reproduction check: original (cloud) vs. independent (Colab) runs. Effective-rank ordering (full < unregularized) is preserved throughout.

Data / Model	Metric	Original	Colab
Structured, full	$\bar{\sigma}$ / rank	1.549 / 1.64	1.551 / 1.61
Structured, unregularized	$\bar{\sigma}$ / rank	0.245 / 6.46	0.244 / 6.48
Digits, full	$\bar{\sigma}$ / rank	1.772 / 1.03	1.418 / 1.05
Digits, unregularized	$\bar{\sigma}$ / rank	0.080 / 3.57	0.079 / 3.65

4.5.4 Robustness Across Seeds

To quantify sensitivity to training stochasticity, the structured and digits experiments were repeated over five random seeds (data content held fixed; the seed varies weight initialization and minibatch order). Table 4.5 and Figure 4.4 report the mean and standard deviation of the collapse indicators. The separation between the full model and the unregularized baseline is consistent across seeds: on both datasets the error bars for effective rank do not overlap, and the direction of every comparison is preserved. The single-seed results reported above are therefore representative rather than incidental. The five-seed means were additionally reproduced on the independent Colab run (e.g. structured full effective rank 1.99 ± 0.25 vs. 1.99 ± 0.24 here).

Table 4.5: Collapse indicators over five seeds (mean $\pm$ std). The full-vs-unregularized ordering holds for every seed on both datasets.

Data	Model	$\bar{\sigma}$	Eff. rank
Structured	Unregularized	0.251 ± 0.005	6.58 ± 0.41
	EdgeVerify (full)	1.449 ± 0.068	1.99 ± 0.24
Digits	Unregularized	0.082 ± 0.004	3.30 ± 0.33
	EdgeVerify (full)	1.464 ± 0.237	1.05 ± 0.02

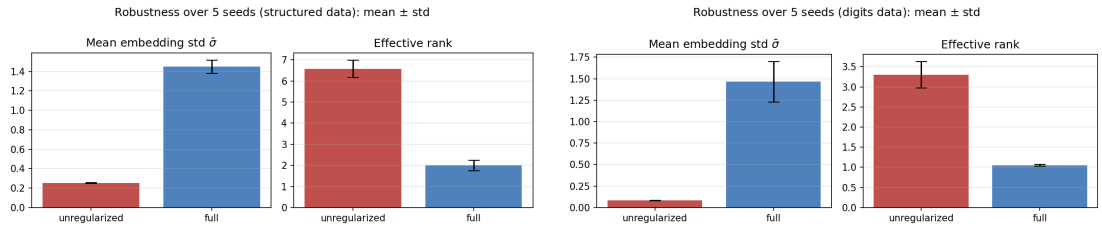


Figure 4.4: Collapse indicators over five seeds (mean $\pm$ std). Error bars for effective rank do not overlap between the two models on either dataset.

4.5.5 Covariance-Weight Ablation

The low effective rank of the full model raises the question of whether it is simply a consequence of an under-weighted covariance term. To test this, the covariance

weight β was swept over four orders of magnitude on the structured dataset with the variance weight fixed ($\alpha = 1.0$); Table 4.6 and Figure 4.5 report the outcome.

Table 4.6: Covariance-weight (β) ablation on structured data ($\alpha = 1.0$, single seed). Increasing β raises effective rank monotonically, but by a small margin.

β	$\bar{\sigma}$	Eff. rank
0.01 (default)	1.551	1.61
0.10	1.567	1.82
1.00	1.567	1.96
10.0	1.471	2.19

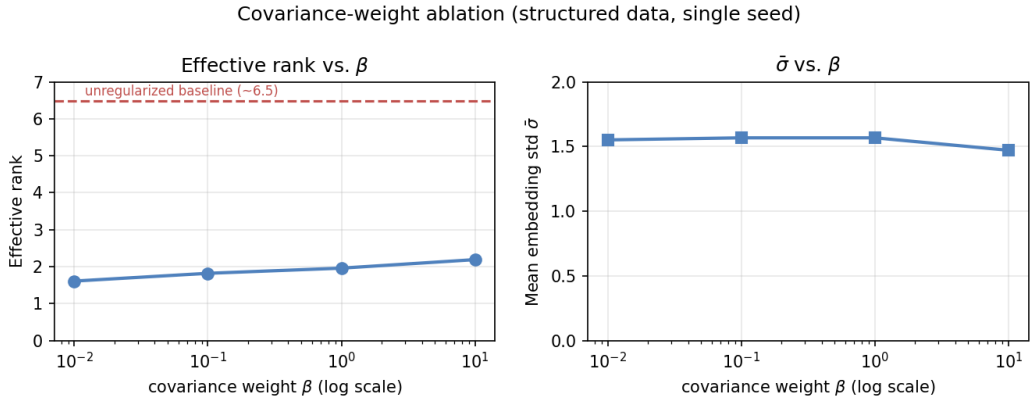


Figure 4.5: Covariance-weight ablation. Effective rank increases monotonically with β but remains far below the unregularized baseline (~ 6.5 , dashed) even at $\beta = 10$; mean embedding standard deviation is essentially unaffected.

The ablation is informative in a negative sense. Increasing β does move effective rank in the expected direction—monotonically, from 1.61 at the default to 2.19 at $\beta = 10$ (a $1000\times$ increase)—confirming that the covariance term is directionally effective. However, the magnitude is small: even at $\beta = 10$ the effective rank (2.19) remains far below both the unregularized baseline (~ 6.5) and the 256-dimensional ceiling, while $\bar{\sigma}$ stays essentially constant. Rank collapse is therefore *not* attributable merely to an under-weighted covariance penalty. A more likely cause is structural: the variance and covariance penalties are applied to *different* tensors (the context embedding s_t and the prediction s_{pred} ; Chapter 3), so the term that sustains variance and the term that decorrelates never act on the same representation. Testing a formulation that applies both penalties to a common embedding is identified as the principal next step (Chapter 5).

4.6 Discussion

On RQ1, the evidence is partial: the variance constraint demonstrably mitigates variance collapse, but at $\beta = 0.01$ the covariance constraint does not yield a high-rank representation, and the unregularized baseline is paradoxically higher-rank—a pattern that holds on the CIFAR-10 natural-image subset as well as on

the smaller datasets. A plausible explanation is that the variance and covariance penalties act on different tensors (the context embedding s_t and the prediction s_{pred} , respectively; Chapter 3) and that the covariance weight is small relative to the variance weight. The covariance-weight ablation (Section 4.5.5) tests this directly and shows that increasing β alone is insufficient to restore rank, which points to the different-tensors structure as the underlying cause rather than mere mis-weighting. On RQ2, the architecture is genuinely lightweight (2.84 MB of parameters, millisecond CPU inference), supporting its suitability for edge deployment in terms of size. On RQ3, the distillation term converges rapidly in all cases; the full objective’s total loss is non-monotonic because the covariance term grows once distillation is satisfied.

Notably, the unregularized baseline attains a far lower total loss ($\sim 10^{-3}$) precisely because, with $\alpha = \beta = 0$, the objective reduces to distillation MSE, which is trivially minimized while the representation quietly loses variance. This is a concrete illustration of why loss magnitude alone is an inadequate proxy for representation quality, and why the collapse indicators are necessary.

4.7 Threats to Validity

The following factors were examined and, where possible, addressed. Run-to-run variance, initially a concern, has been *quantified*: the five-seed study (Section 4.5.4) shows the finding is stable and the full-vs-unregularized separation holds for every seed. Two factors remain open. First, the benchmark coverage is partial: two of the three datasets are non-standard, and the CIFAR-10 evaluation used a 2000-image subset with a reduced epoch budget rather than the full benchmark. Second, all measurements are CPU-only, so latency and memory would differ on target embedded hardware, and the encoder is small relative to the transformer backbones typical of the JEPA literature. Addressing these is the subject of Chapter 5.

Chapter 5

Conclusion and Future Work

5.1 Summary

This thesis presented **EdgeVerify**, a Joint-Embedding Predictive Architecture designed for localized execution on edge and Dew computing nodes. The architecture combines an EMA-updated target encoder, an action-conditioned latent predictor, and a VICReg-inspired variance/covariance objective applied within a local, layer-wise distillation loss. By predicting in an abstract latent space rather than reconstructing raw inputs, and by bounding the backpropagation graph through stop-gradient firebreaks, the design targets the two coupled obstacles to on-device learning identified in Chapter 1: strict memory budgets and representational collapse.

5.2 Contributions

The work contributes: (i) the EdgeVerify architecture and its action-conditioned prediction formulation; (ii) a negative-free, stable local loss that regularizes both the encoder output (variance) and the predictor output (covariance); and (iii) a proof-of-concept PyTorch implementation together with a reproducible protocol for evaluating convergence, collapse, latency, and memory footprint.

5.3 Limitations

The principal limitation of the present work is that empirical validation is preliminary. The evaluation in Chapter 4 covered structured, digits, and a 2000-image CIFAR-10 subset, with CPU-only measurement; the structured and digits results were shown to be stable across five seeds, but a full-scale benchmark run on target hardware remains outstanding. Within those limits, the evaluation surfaced a substantive finding, reproduced across two independent environments: while the variance term prevents variance collapse, the covariance term at its current weight ($\beta = 0.01$) fails to prevent dimensional (rank) collapse, and the unregularized baseline was in fact higher-rank—a pattern that held on the CIFAR-10 natural images as well as the smaller datasets. A covariance-weight ablation (Section 4.5.5) established that this is not resolved by increasing β alone—effective rank rises only marginally—indicating that the collapse-prevention claim is only

partially supported and that a structural change to the objective, rather than re-weighting, is required. In addition, the convolutional encoder used in the proof-of-concept is small relative to the transformer backbones typical of the JEPA literature, so conclusions drawn here may not transfer directly to larger models. The “multi-agent” aspect suggested by the title is, at present, realized only as the multi-module (context/target/predictor) structure of a single node; coordination across multiple physical agents is not yet implemented.

5.4 Future Work

Several directions follow directly:

- **Shared-embedding regularization.** The covariance-weight ablation (Section 4.5.5) showed that increasing β alone does not restore effective rank. The most direct next step, suggested by that result, is to apply the variance and covariance penalties to a *common* embedding—rather than to s_t and s_{pred} separately—so that the term sustaining variance and the term enforcing decorrelation act on the same representation.
- **Full-scale and on-device evaluation.** Run the existing protocol on the complete CIFAR-10 benchmark (and larger datasets) and on a defined edge device, to replace the subset results and CPU-only measurements with full-scale and hardware-accurate figures.
- **Backbone scaling.** Evaluate whether the local objective remains stable with larger or transformer-based encoders.
- **True multi-agent extension.** Extend the single-node design to coordinated learning across multiple edge nodes, connecting the framework to decentralized-learning approaches such as federated learning [13].

References

- [1] W. Shi, J. Cao, Q. Zhang, Y. Li, and L. Xu, “Edge computing: Vision and challenges,” *IEEE Internet of Things Journal*, vol. 3, no. 5, pp. 637–646, 2016.
- [2] Y. Wang, “Definition and categorization of dew computing,” *Open Journal of Cloud Computing (OJCC)*, vol. 3, no. 1, pp. 1–7, 2016.
- [3] Y. LeCun, “A path towards autonomous machine intelligence version 0.9. 2, 2022-06-27,” *Open Review*, vol. 62, 2022.
- [4] M. Assran, Q. Duval, I. Misra, *et al.*, “Self-supervised learning from images with a joint-embedding predictive architecture,” *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023.
- [5] A. Bardes, J. Ponce, and Y. LeCun, “VICReg: Variance-invariance-covariance regularization for self-supervised learning,” in *International Conference on Learning Representations (ICLR)*, 2022.
- [6] T. Chen, S. Kornblith, M. Norouzi, and G. Hinton, “A simple framework for contrastive learning of visual representations,” in *International Conference on Machine Learning (ICML)*, 2020.
- [7] J.-B. Grill, F. Strub, F. Altché, *et al.*, “Bootstrap your own latent: A new approach to self-supervised learning,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.
- [8] X. Chen and K. He, “Exploring simple siamese representation learning,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2021.
- [9] J. Zbontar, L. Jing, I. Misra, Y. LeCun, and S. Deny, “Barlow twins: Self-supervised learning via redundancy reduction,” in *International Conference on Machine Learning (ICML)*, 2021.
- [10] K. He, X. Chen, S. Xie, Y. Li, P. Dollár, and R. Girshick, “Masked autoencoders are scalable vision learners,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022.
- [11] G. Hinton, O. Vinyals, and J. Dean, “Distilling the knowledge in a neural network,” *arXiv preprint arXiv:1503.02531*, 2015.
- [12] M. Caron, H. Touvron, I. Misra, *et al.*, “Emerging properties in self-supervised vision transformers,” in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2021.

- [13] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y. Arcas, “Communication-efficient learning of deep networks from decentralized data,” in *Artificial Intelligence and Statistics (AISTATS)*, 2017.